

Omise Payment Gateway integrated with JAVA

1. ทำ Manual JAR โดยการดาวน์โหลด JAR ทั้งหมดจาก Maven Central แล้วเอาไปเพิ่มไว้ใน Folder ตาม path ดังนี้ \WEB-INF\lib ของโปรเจกต์ :

- omise-java-5.2.0.jar
- kotlin-stdlib-jdk8-1.9.10.jar
- okhttp-4.9.3.jar, okio-2.10.0.jar
- jackson-core-2.9.8.jar, jackson-databind-2.9.8.jar, jackson-annotations-2.9.0.jar, jackson-datatype-jsr310-2.9.8.jar

2. เก็บ Secret Key และ Public Key ไว้ในไฟล์ application.properties หรือเป็น environment variable :

application.properties

omise.public-key=pkey_test_XXXXXXXXXXXX

omise.secret-key=skey_test_XXXXXXXXXXXX

3. สร้างไฟล์ OmiseChargeService.java สำหรับสร้าง Client และ Charge :

```
package com.cubesofttech.service;

import co.omise.Client;
import co.omise.ClientException;
import co.omise.models.Charge;
import co.omise.requests.Request;
import co.omise.models.Source;
import co.omise.models.SourceType;
import org.jfree.util.Log;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import javax.annotation.PostConstruct;
```

@Service

```
public class OmiseChargeService {
```

```
    @Value("${omise.secret-key}")
```

```
    private String secretKey;
```

```
    private Client client;
```

```
    @PostConstruct
```

```
    public void init() {
```

```
        try {
```

```
            this.client = new Client.Builder().secretKey(secretKey).build();
```

```
        } catch (ClientException e) {
```

```
            e.printStackTrace();
```

```
            throw new IllegalStateException("Cannot initialize Omise client", e);
```

```
        }
```

```
    }
```

```
    /**
```

```
     * สร้าง Charge จากบัตรเครดิต (token จาก Omise.js)
```

```
    */
```

```
    public Charge createCardCharge(String token, int amount) throws Exception {
```

```
        Request<Charge> req = new
```

```
        Charge.CreateRequestBuilder().amount(amount).currency("thb").card(token).build();
```

```
        return client.sendRequest(req);
```

```
    }
```

```
    /**
```

```
     * สร้าง Charge สำหรับ PromptPay source ที่มีอยู่แล้ว
```

```
    */
```

```
    public Charge createPromptpayCharge(String sourceId, int amount) throws Exception {
```

```
        Request<Charge> req = new
```

```
        Charge.CreateRequestBuilder().amount(amount).currency("thb").source(sourceId).build();
```

```
        return client.sendRequest(req);
```

```
    }
```

```
    /**
```

```
     * สร้าง PromptPay Source (คืน sourceId ไปให้ frontend สแกน QR code)
```

```

        */

    public Source createPromptpaySource(int amount) throws Exception {
        Request<Source> req = new Source.CreateRequestBuilder()
            .type(SourceType.PromptPay)
            .amount(amount)
            .currency("thb")
            .build();
        return client.sendRequest(req);
    }
}

```

4. เขียน javascript กำหนด configure ของ Omise ในหน้า xxx.jsp :

-ทำการ import script ผ่าน CDNs ของ Omise ดังนี้

```
<script type="text/javascript" src="https://cdn.omise.co/omise.js"></script>
```

วิธีการเรียกใช้ Omise Methods สำหรับการสร้าง Source

-กำหนด Public key แบบ one-time-use ดังนี้

```
Omise.setPublicKey("pkey_test_XXXXXXXXXXXXX");
```

-กำหนด Public key แบบ OmiseCard ดังนี้

```

OmiseCard.configure({

    publicKey: "pkey_test_XXXXXXXXXXXXX"

});

```

- การสร้าง Source จากฝั่ง backend สามารถทำได้โดยการเรียกใช้เมธอด createXXXSource ที่อยู่ใน Service

- การสร้าง Source/Token จากฝั่ง frontend (ดูตัวอย่างได้จาก <https://docs.omise.co/omise-js>)

```

sourceParameters = {
    "amount": 12345,
    "currency": "THB",

```

```

    "phone_number": "0123456789"
}

Omise.createSource("promptpay", sourceParameters, function(statusCode, response) {
// response["id"] is source identifier
console.log(response)
});

```

5. สร้าง Action Controller :

```

public String charge() {
    try {
        String source = request.getParameter("source");
        String amount = request.getParameter("amount");
        //สร้าง Charge โดยส่ง parameter ตามที่ payment method แต่ละตัวต้องการ
        Charge charge = omiseService.createXXXCharge(source, Integer.parseInt(amount));
        return SUCCESS;
    } catch (Exception e) {
        return ERROR;
    }
}

```

6. WebHook

-สร้าง action ไว้สำหรับเชื่อม Webhook endpoint

```
<action name="omiseWebhook" class="com.cubesofttech.actionfront.OmiseWebhookAction" method="execute">
```

```
    <result name="none" type="httpheader" />
```

```
</action>
```

-สร้าง OmiseWebhookAction.java

```

public String execute() {

    try {

        String payload;

        try (InputStream in = request.getInputStream()) {

```

```

        payload = readBody(in, request.getCharacterEncoding());
    }

    ObjectMapper mapper = new ObjectMapper();

    JsonNode root = mapper.readTree(payload);

    ObjectWriter ow = new ObjectMapper().writer().withDefaultPrettyPrinter();

    String json = ow.writeValueAsString(root);

    String event = root.path("key").asText();

    log.debug("Got event: " + event);

    // TODO: ยืนยันกับ Omise API

response.setStatus(200);

    response.getWriter().write("ok");

    return NONE;

} catch (Exception e) {

    e.printStackTrace();

    try {

        response.setStatus(500);

        response.getWriter().write("error");

    } catch (Exception ignore) {}

    return NONE;

}

private String readBody(InputStream in, String charset) throws IOException {

    ByteArrayOutputStream buffer = new ByteArrayOutputStream();

    byte[] data = new byte[1024];

    int nRead;

    while ((nRead = in.read(data, 0, data.length)) != -1) {

        buffer.write(data, 0, nRead);

```

```
}
```

```
buffer.flush();
```

```
return new String(buffer.toByteArray(), charset != null ? charset : "UTF-8");
```

```
}
```

-กำหนด Webhook endpoint ที่ฝั่ง omise dashboard(<https://dashboard.omise.co/v2/settings/webhooks>)

<https://<your-domain>/omiseWebhook>